c1669bef-ca65-42e3-bfa5-3a3de8924609

<prev_user>

<request>

You are working on an issue in the 'intel/torch-xpu-ops' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/torch-xpu-ops/torch-xpu-ops (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-11T04:30:05.293+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section details on the original issue you should resolve

<issue_title>[UT] "RuntimeError: iter.device(arg).is_xpu()" in test_torch_xpu.py</issue_title>
<issue_description>## Summary
RuntimeError in XPU kernel Loops.h when torch.addcmul is called with a CPU scalar tensor (use_cpu_scalar=True). The XPU kernel asserts that all arguments are on XPU, but the scalar tensor remains on CPU.

Test Info
- **Test Type:** ut
- **Category:**
- **Dependency:**
- **Platform:** xpu

Failed Tests
-
`test_torch_xpu.py::TestVitalSignsCudaXPU::test_cuda_vitals_gpu_only_xpu`
-

`test_torch_xpu.py::TestTorchDeviceTypeXPU::test_addcmul_use_cpu_scalar_True_xpu_complex128`
-
`test_torch_xpu.py::TestTorchDeviceTypeXPU::test_addcmul_use_cpu_scalar_True_xpu_complex64`
-
`test_torch_xpu.py::TestTorchDeviceTypeXPU::test_addcmul_use_cpu_scalar_True_xpu_float32`
-
`test_torch_xpu.py::TestTorchDeviceTypeXPU::test_addcmul_use_cpu_scalar_True_xpu_float64`
-
`test_torch_xpu.py::TestTorchDeviceTypeXPU::test_addcmul_use_cpu_scalar_True_xpu_int16`
-
`test_torch_xpu.py::TestTorchDeviceTypeXPU::test_addcmul_use_cpu_scalar_True_xpu_int32`
-
`test_torch_xpu.py::TestTorchDeviceTypeXPU::test_addcmul_use_cpu_scalar_True_xpu_int64`
-
`test_torch_xpu.py::TestTorchDeviceTypeXPU::test_addcmul_use_cpu_scalar_True_xpu_int8`
-
`test_torch_xpu.py::TestTorchDeviceTypeXPU::test_addcmul_use_cpu_scalar_True_xpu_uint8`

Error Log
```
_____
TestTorchDeviceTypeXPU.test_addcmul_use_cpu_scalar_True_xpu_uint8 _______

self = <xpu.test_torch_xpu.TestTorchDeviceTypeXPU testMethod=test_addcmul_use_cpu_scalar_True_xpu_uint8>
device = 'xpu:0', dtype = torch.uint8, use_cpu_scalar = True

>       actual = torch.addcmul(a, b, c, value=alpha)
E       RuntimeError: iter.device(arg).is_xpu() INTERNAL ASSERT FAILED at "/pytorch/third_party/torch-xpu-ops/src/ATen/native/xpu/sycl/Loops.h":626, please report a bug to PyTorch. argument 3: expected an XPU device but found cpu
```

Reproducer
```bash
PYTORCH_TEST_WITH_SLOW=1 python test/xpu/test_torch_xpu.py
```

TestTorchDeviceTypeXPU.test_addcmul_use_cpu_scalar_True_xpu_uint
8
```

## Commit Scope


## Root Cause Analysis
_Pending triage_

## Proposed Fix Strategy
_Pending triage_

## Action Items
- [x] 🔍 Issue formatted (Discovery Agent)

- [ ] 🧠 Root cause identified (Triage Agent)

- [ ] 🔧 Fix implemented (Fix Agent)

- [ ] ✔ Fix verified locally
- [ ] 📋 PR proposed
- [ ] 👀 Human review
- [ ] 🎉 PR merged

## Context
https://github.com/intel/torch-xpu-ops/pull/2559

## Environment


## Original Issue

</issue_description>

<agent_instructions>Please open a PR and fix the issue.</
agent_instructions>

## Comments on the Issue (you are @copilot in this section)

<comments>
<comment_new><author>@CuiYifeng</author>
To add support for CPU scalar in `addcmul`.</comment_new>
<comment_new><author>@chuanqi129</author>
@libohao1201 please double check this file whether enabled in
CI</comment_new>
```

</comments>
</problem_statement>

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

<system_reminder>You have the capability to call multiple tools in a single response.
For maximum efficiency, whenever you need to perform multiple independent operations, ALWAYS call tools simultaneously whenever the actions can be done in parallel rather than sequentially (e.g. multiple reads/edits to different files). Especially when exploring repository, searching, reading files, viewing directories, validating changes. For example, you can read 3 different files in parallel, or edit different files in parallel. However, if some tool calls depend on previous calls to inform dependent values like the parameters, do NOT call these tools in parallel and instead call them sequentially (e.g. reading shell output from a previous command should be sequential as it requires the sessionID).</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

The implementation is complete. Here is the summary:

Agent Summary
- **What I found:** The `addcmul_kernel` in `PointwiseOpsKernels.cpp` did not handle the case where `tensor2` (TensorIterator argument index 3) is a CPU scalar tensor. The `gpu_kernel` function asserts that all operands are on XPU, causing a `RuntimeError` when `tensor2` is a CPU scalar.
- **What I changed:**
  - `src/ATen/native/xpu/sycl/PointwiseOpsKernels.cpp`: Added `AddcmulCpuScalarFunctor` and `AddcmulComplexCpuScalarFunctor` (2-argument functors capturing the CPU scalar value), and updated `addcmul_kernel` to d `iter.is_cpu_scalar(3)`, extract the scalar value, remove the operand, and dispatch using the 2-argument functor.
  - `test/repro/test_addcmul_cpu_scalar.py`: Added reproducer test covering all affected dtypes.

- **Test file:** test/repro/test_addcmul_cpu_scalar.py
- **Branch:** agent/fix-addcmul-cpu-scalar

(Note: branch name may appear as `copilot/fix-runtimeerror-xpu-kernel` in git)

- **PR body draft:**

Fix RuntimeError in addcmul_kernel when tensor2 is a CPU scalar on XPU

The XPU `addcmul_kernel` failed with `RuntimeError: iter.device(arg).is_xpu()` when `tensor2` was a CPU scalar tensor. The `gpu_kernel` function asserts all operands are on XPU, but CPU scalar tensors remain on CPU. Added `AddcmulCpuScalarFunctor` and `AddcmulComplexCpuScalarFunctor` (2-argument functors that capture the CPU scalar value as a constant), and updated `addcmul_kernel` to detect `iter.is_cpu_scalar(3)`, extract the scalar value, remove operand 3, and dispatch the 2-argument functor. This matches the pattern used by `opmath_gpu_kernel_with_scalars` for handling CPU scalars in binary kernels.

Test: test/repro/test_addcmul_cpu_scalar.py

PyTorch test: `test/xpu/test_torch_xpu.py::TestTorchDeviceTypeXPU::test_addcmul_use_cpu_scalar_True_xpu_*`

- **Open questions / risks:** None — the fix is targeted and follows the existing CPU scalar handling pattern in the XPU Loops.h infrastructure.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'intel/torch-xpu-ops' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/torch-xpu-ops/torch-xpu-ops (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-11T06:31:26.251+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
[XPU] Fix RuntimeError in addcmul when tensor2 is a CPU scalar tensor
</pr_title>

<pr_description>
`torch.addcmul` on XPU raised `RuntimeError: iter.device(arg).is_xpu()` whenever `tensor2` was a CPU scalar tensor, because `gpu_kernel` asserts all operands are on XPU with no fallback for CPU scalars.

Changes

- **`src/ATen/native/xpu/sycl/PointwiseOpsKernels.cpp`**
  - Add `AddcmulCpuScalarFunctor` and `AddcmulComplexCpuScalarFunctor`: 2-argument functors that capture the CPU scalar value of `tensor2` as a compile-time constant, eliminating it from the iterator before dispatch.
  - In `addcmul_kernel`, detect `iter.is_cpu_scalar(3)`, extract the scalar via `iter.scalar_value<>`, call `iter.remove_operand(3)`, then dispatch the 2-arg functor — matching the pattern used by `opmath_gpu_kernel_with_scalars` for binary CPU scalar handling.
  - Use `AT_DISPATCH_ALL_TYPES_AND2` (not `_COMPLEX`) in the non-complex CPU-scalar branch, since complex types are already handled by the preceding `if` block.

- **`test/repro/test_addcmul_cpu_scalar.py`**: Reproducer covering all affected dtypes (`float32`, `float64`, `complex64`, `complex128`, `int8`, `int16`, `int32`, `int64`, `uint8`).

```python
# Previously raised RuntimeError on XPU
a = torch.rand(3, 3, dtype=torch.float32, device="xpu")
b = torch.rand(3, 3, dtype=torch.float32, device="xpu")
c = torch.tensor(2.0, dtype=torch.float32, device="cpu")  # CPU scalar
out = torch.addcmul(a, b, c, value=0.5)  # now works correctly
```

Test: `test/xpu/
test_torch_xpu.py::TestTorchDeviceTypeXPU::test_addcmul_use_cpu_
scalar_True_xpu_*`
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_new>
<comment_id>4418095470</comment_id>
<author>@Stonepia</author>
@copilot fix the ci failue
</comment_new>

</pr_comments>

</comments>

The last **3** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<system_reminder>
You have the capability to call multiple tools in a single response. For maximum efficiency, whenever you need to perform multiple independent operations, ALWAYS call tools simultaneously whenever the actions can be done in parallel rather than sequentially (e.g. multiple reads/edits to different files). Especially when exploring repository, searching, reading files, viewing directories, validating changes. For example, you can read 3 different files in parallel, or edit different files in parallel. However, if some tool calls depend on previous calls to inform dependent values like the parameters, do NOT call these tools in parallel and instead call them sequentially (e.g. reading shell output from a previous command should be sequential as it requires the sessionID).
</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.